

REPORTE DE ANÁLISIS ESTÁTICO

EcoRoute — Resultados de Herramientas de Análisis Estático

1. INTRODUCCIÓN

1.1 Propósito

Este documento presenta los resultados del análisis estático del código fuente del sistema EcoRoute. El análisis estático examina el código sin ejecutarlo, identificando problemas de calidad, vulnerabilidades de seguridad, código duplicado, violaciones de estilo y métricas de complejidad antes de que se conviertan en defectos en tiempo de ejecución.

1.2 Herramientas Utilizadas

Herramienta	Versión	Propósito	Comando
flake8	6.x	Verificación de estilo PEP8 y errores sintácticos	flake8 . --max-line-length=88
pylint	3.x	Análisis profundo de calidad: errores, advertencias, convenciones y refactorización	pylint environment/ agent/ training/ evaluation/ config/
black	23.x	Verificación de formato consistente del código	black --check .
bandit	1.7.x	Análisis de seguridad: detección de vulnerabilidades comunes en Python	bandit -r . -f json
radon	6.x	Métricas de complejidad ciclomática y mantenibilidad	radon cc . -a -s
mypy	1.x	Verificación estática de tipos (type hints)	mypy . --ignore-missing-imports
SonarQube	Community	Análisis integral de calidad con dashboard visual	Integración via sonar-scanner

1.3 Cuándo Ejecutar el Análisis

Momento	Herramientas	Automatización
Antes de cada commit	black, flake8	pre-commit hook
En cada Pull Request	flake8, pylint, bandit, mypy	CI pipeline
Al cierre de cada sprint	radon, SonarQube completo	Manual
Al liberar cada versión	Todas las herramientas	CI/CD pipeline

1.4 Cómo Instalar las Herramientas

bash

pip install flake8 pylint black bandit radon mypy

pip install pre-commit

pre-commit install

1.5 Estado del Reporte

Campo	Valor
Versión del sistema analizada	EcoRoute v1.0.0
Estado	Plantilla — completar con resultados reales tras ejecutar las herramientas
Líneas de código totales (LOC)	Por determinar
Fecha de análisis	Por determinar

2. CONFIGURACIÓN DE LAS HERRAMIENTAS

2.1 Archivo .flake8

ini

[flake8]

max-line-length = 88

exclude =

.git,

__pycache__,

htmlcov,

results/

ignore =

E203,

W503

per-file-ignores =

```
tests/*: F401
```

2.2 Archivo .pylintrc (fragmento clave)

```
ini
```

```
[MESSAGES CONTROL]
```

```
disable =
```

```
    C0114, # missing-module-docstring (módulos de prueba)
```

```
    R0903 # too-few-public-methods (Value Objects)
```

```
[FORMAT]
```

```
max-line-length = 88
```

```
[DESIGN]
```

```
max-args = 8
```

```
max-locals = 15
```

```
max-returns = 6
```

```
max-branches = 12
```

2.3 Archivo pyproject.toml (black)

```
toml
```

```
[tool.black]
```

```
line-length = 88
```

```
target-version = ['py310']
```

```
exclude = ""
```

```
/(
```

```
    \.git
```

```
    | htmlcov
```

```
    | results
```

```
)/
```

```
""
```

2.4 Archivo .bandit

yaml

skips: []

exclude_dirs:

- tests/
- htmlcov/
- results/

3. RESULTADOS DE FLAKE8 — ESTILO Y SINTAXIS

3.1 Resumen de Resultados

Completar tras ejecutar: `flake8 . --max-line-length=88 --statistics`

Módulo	Errores E	Advertencias W	Total	Estado
config/config_loader.py	—	—	—	Pendiente
environment/sumo_env.py	—	—	—	Pendiente
environment/emissions.py	—	—	—	Pendiente
environment/reward.py	—	—	—	Pendiente
agent/dqn_agent.py	—	—	—	Pendiente
agent/replay_buffer.py	—	—	—	Pendiente
agent/network.py	—	—	—	Pendiente
training/trainer.py	—	—	—	Pendiente
training/baselines.py	—	—	—	Pendiente
evaluation/logger.py	—	—	—	Pendiente
evaluation/visualizer.py	—	—	—	Pendiente
TOTAL	—	—	—	Pendiente

3.2 Criterio de Aceptación

Criterio	Umbral	Estado
Errores E (errores de sintaxis o estilo crítico)	0	Pendiente

Advertencias W (advertencias de estilo)	≤ 10 en todo el proyecto	Pendiente
Módulos con más de 5 issues	0 módulos	Pendiente

3.3 Comando de Ejecución

bash

flake8 . --max-line-length=88 --statistics --count

4. RESULTADOS DE PYLINT — CALIDAD DE CÓDIGO

4.1 Escala de Puntuación Pylint

Pylint asigna una puntuación de 0 a 10 basada en la cantidad y severidad de los issues encontrados.

Puntuación Clasificación Acción requerida

9.0 — 10.0	Excelente	Ninguna
8.0 — 8.9	Buena	Revisar issues menores
7.0 — 7.9	Aceptable	Resolver antes de la siguiente versión
6.0 — 6.9	Regular	Resolver en el sprint actual
< 6.0	Inaceptable	Bloquea el cierre del sprint

4.2 Resultados por Módulo

Completar tras ejecutar: `pylint environment/ agent/ training/ evaluation/ config/`

Módulo	Puntuación	Error es (E)	Advertencias (W)	Convenciones (C)	Refactorización (R)	Estado
config/config_loader.py	—/10	—	—	—	—	Pendiente
environment/sumo_env.py	—/10	—	—	—	—	Pendiente
environment/emissions.py	—/10	—	—	—	—	Pendiente
environment/reward.py	—/10	—	—	—	—	Pendiente

agent/dqn_agent.py	—/10	—	—	—	—	Pendiente
agent/replay_buffer.py	—/10	—	—	—	—	Pendiente
agent/network.py	—/10	—	—	—	—	Pendiente
training/trainer.py	—/10	—	—	—	—	Pendiente
training/baselines.py	—/10	—	—	—	—	Pendiente
evaluation/logger.py	—/10	—	—	—	—	Pendiente
evaluation/visualizer.py	—/10	—	—	—	—	Pendiente
PUNTUACIÓN GLOBAL	—/10	—	—	—	—	Pendiente

4.3 Criterio de Aceptación Pylint

Criterio	Umbral	Estado
Puntuación global del proyecto	$\geq 8.0 / 10$	Pendiente
Errores (E) en módulos de producción	0	Pendiente
Módulos con puntuación < 7.0	0 módulos	Pendiente

4.4 Issues Conocidos Esperados

Los siguientes issues de pylint son conocidos y aceptados con justificación:

Código pylint	Descripción	Módulo	Justificación
R0903	too-few-public-methods	Value Objects	Los Value Objects por diseño DDD tienen pocos métodos públicos
C0114	missing-module-docstring	tests/	Los módulos de prueba no requieren docstring de módulo
W0212	protected-access	tests/	Las pruebas unitarias acceden a métodos protegidos por diseño

4.5 Comando de Ejecución

bash

```
pylint environment/ agent/ training/ evaluation/ config/ \
```

```
--output-format=text \
```

```
--reports=yes
```

5. RESULTADOS DE BANDIT — SEGURIDAD

5.1 Clasificación de Vulnerabilidades Bandit

Severidad	Confianza	Acción requerida
HIGH + HIGH	Crítico	Corregir inmediatamente antes del merge
HIGH + MEDIUM	Alto	Corregir en el sprint actual
MEDIUM + HIGH	Alto	Corregir en el sprint actual
MEDIUM + MEDIUM	Medio	Corregir antes de la versión de release
LOW + cualquiera	Bajo	Evaluar y documentar si se acepta el riesgo

5.2 Resumen de Resultados

Completar tras ejecutar: `bandit -r . -f json -o bandit_report.json`

Categoría	Cantidad encontrada	Críticos	Altos	Medios	Bajos
Inyección de código	—	—	—	—	—
Uso de funciones inseguras	—	—	—	—	—
Manejo inseguro de archivos	—	—	—	—	—
Otros	—	—	—	—	—
TOTAL	—	—	—	—	—

5.3 Issues Conocidos Esperados en EcoRoute

Código bandit	Descripción	Módulo	Justificación
B301	pickle usage	agent/dqn_agent.py	PyTorch usa pickle internamente para guardar checkpoints. El archivo .pth solo se carga desde rutas locales configuradas por el investigador. Riesgo aceptado y documentado.

B311	random usage	agent/dqn_agent.py	El módulo random se usa para la política epsilon-greedy. No es un contexto criptográfico. Riesgo aceptado.
------	--------------	--------------------	--

5.4 Criterio de Aceptación Bandit

Criterio	Umbral	Estado
Issues HIGH severity + HIGH confidence	0	Pendiente
Issues HIGH severity + MEDIUM confidence	0 no justificados	Pendiente
Issues documentados y aceptados	Requieren justificación escrita	Pendiente

5.5 Comando de Ejecución

bash

```
bandit -r environment/ agent/ training/ evaluation/ config/ \
-f json \
-o results/bandit_report.json \
--severity-level medium
```

6. RESULTADOS DE MYPY — VERIFICACIÓN DE TIPOS

6.1 Resumen de Resultados

Completar tras ejecutar: `mypy . --ignore-missing-imports --strict`

Módulo	Errores de tipo	Advertencias	Estado
config/config_loader.py	—	—	Pendiente
environment/sumo_env.py	—	—	Pendiente
environment/emissions.py	—	—	Pendiente
environment/reward.py	—	—	Pendiente
agent/dqn_agent.py	—	—	Pendiente
agent/replay_buffer.py	—	—	Pendiente
agent/network.py	—	—	Pendiente
training/trainer.py	—	—	Pendiente

training/baselines.py	—	—	Pendiente
evaluation/logger.py	—	—	Pendiente
evaluation/visualizer.py	—	—	Pendiente
TOTAL	—	—	Pendiente

6.2 Criterio de Aceptación Mypy

Criterio

Umbral Estado

Errores de tipo en módulos de producción 0 Pendiente

Cobertura de type hints en módulos críticos $\geq 80\%$ Pendiente

6.3 Comando de Ejecución

bash

mypy environment/ agent/ training/ evaluation/ config/ \

--ignore-missing-imports \

--show-error-codes \

--pretty

7. RESUMEN DE SONARQUBE

7.1 Cómo Configurar SonarQube para EcoRoute

bash

Instalar sonar-scanner

pip install pysonar-scanner

Crear sonar-project.properties en la raíz del proyecto

cat > sonar-project.properties << EOF

sonar.projectKey=ecoroute

sonar.projectName=EcoRoute

sonar.projectVersion=1.0.0

sonar.sources=environment,agent,training,evaluation,config

sonar.tests=tests

sonar.python.coverage.reportPaths=coverage.xml

```
sonar.python.pylint.reportPaths=pylint_report.txt
```

```
EOF
```

```
# Ejecutar análisis
```

```
sonar-scanner
```

7.2 Métricas Objetivo en SonarQube

Métrica SonarQube	Umbral objetivo	Descripción
Reliability Rating	A	0 bugs detectados
Security Rating	A	0 vulnerabilidades
Maintainability Rating	A	Deuda técnica < 5% del tiempo de desarrollo
Coverage	$\geq 80\%$	Cobertura de pruebas automatizadas
Duplications	$\leq 3\%$	Código duplicado en el proyecto
Code Smells	≤ 20	Issues de mantenibilidad acumulados
Bugs	0	Errores detectados por análisis estático
Vulnerabilities	0	Vulnerabilidades de seguridad
Security Hotspots	0 no revisados	Puntos de atención de seguridad

7.3 Resultados de SonarQube

Completar tras ejecutar sonar-scanner y consultar el dashboard

Métrica	Valor objetivo	Valor obtenido	Rating	Estado
Reliability Rating	A	—	—	Pendiente
Security Rating	A	—	—	Pendiente
Maintainability Rating	A	—	—	Pendiente
Coverage	$\geq 80\%$	—%	—	Pendiente
Duplications	$\leq 3\%$	—%	—	Pendiente
Code Smells	≤ 20	—	—	Pendiente
Bugs	0	—	—	Pendiente
Vulnerabilities	0	—	—	Pendiente

Lines of Code	—	—	—	Pendiente
---------------	---	---	---	-----------

8. CRITERIO DE CALIDAD GLOBAL — QUALITY GATE

El proyecto supera el Quality Gate cuando cumple **todos** los criterios:

Criterio	Herramienta	Umbral	Estado
0 errores de sintaxis	flake8	E = 0	Pendiente
Puntuación pylint global	pylint	$\geq 8.0/10$	Pendiente
0 vulnerabilidades críticas	bandit	HIGH+HIGH = 0	Pendiente
0 errores de tipo	mypy	0 errores	Pendiente
Cobertura de pruebas	pytest-cov	$\geq 80\%$	Pendiente
Reliability SonarQube	SonarQube	Rating A	Pendiente
Security SonarQube	SonarQube	Rating A	Pendiente
Código duplicado	SonarQube	$\leq 3\%$	Pendiente